

Ampliación exposición TFG (Ejemplo de retropropagación)

Red neuronal con una única capa oculta

Óscar Climent Ivorra

11 de junio de 2026

Objetivo del ejemplo

Idea principal

El objetivo es calcular de forma eficiente el gradiente del error asociado a una observación:

$$\nabla R_n(w), \quad \text{es decir, } \frac{\partial R_n}{\partial w_{ji}}$$

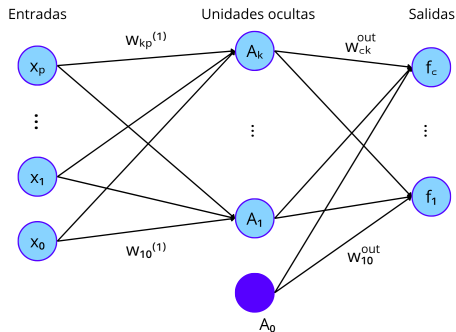
para poder actualizar los pesos durante el entrenamiento de la red.

- Primero se propaga la información hacia delante.
- Después se calcula el error en la capa de salida.
- Finalmente, el error se propaga hacia atrás para obtener las derivadas.

Arquitectura considerada

- Entrada: vector $X_n \in \mathbb{R}^P$.
- Una capa oculta con K unidades.
- Salida con C clases.
- Se introduce una unidad adicional para incluir el sesgo:

$$x_{n0} = 1, \quad A_0^{(1)} = 1.$$



Datos, activaciones y función de error

Para la observación n del conjunto de entrenamiento:

$$X_n = (x_{n1}, x_{n2}, \dots, x_{np}) \in \mathbb{R}^p.$$

La salida real se representa mediante codificación 1-de- K :

$$Y_{nc} = \begin{cases} 1, & \text{si la clase verdadera es } c, \\ 0, & \text{en caso contrario.} \end{cases}$$

Elecciones del ejemplo

$$g(a) = \tanh(a), \quad f_c(X_n, w) = \frac{e^{a_c^{\text{out}}(X_n, w)}}{\sum_{l=1}^C e^{a_l^{\text{out}}(X_n, w)}} \quad \text{Función de activación Softmax,}$$

$$R_n(w) = - \sum_{c=1}^C Y_{nc} \ln(f_c(X_n, w)) \quad \text{Entropía Cruzada.}$$

Paso 1: propagación hacia delante

Se calcula primero la información en la capa oculta:

$$a_j^{(1)} = \sum_{i=0}^p w_{ji}^{(1)} x_{ni}, \quad j = 1, \dots, K,$$

$$A_j^{(1)} = g(a_j^{(1)}), \quad j = 1, \dots, K.$$

Después, se calcula la salida de la red:

$$a_c^{out} = \sum_{h=0}^K w_{ch}^{out} A_h^{(1)}, \quad c = 1, \dots, C,$$

$$f_{cn} = \frac{e^{a_c^{out}}}{\sum_{l=1}^C e^{a_l^{out}}}, \quad c = 1, \dots, C.$$

Interpretación

f_{cn} es la probabilidad predicha de que la observación n pertenezca a la clase c .

Paso 2: error en la capa de salida

En clasificación multiclase, al combinar **softmax** con **entropía cruzada**, se obtiene una expresión especialmente simple:

$$\delta_c^{out} = \frac{\partial R_n}{\partial a_c^{out}}.$$

Partiendo de

$$R_n(w) = - \sum_{l=1}^C Y_{nl} \ln \left(\frac{e^{a_l^{out}}}{\sum_{r=1}^C e^{a_r^{out}}} \right),$$

se llega a

$$\boxed{\delta_c^{out} = f_{cn} - Y_{nc}}, \quad c = 1, \dots, C.$$

Interpretación

El término de error en la salida es la diferencia entre la probabilidad predicha y el valor real codificado como 0 o 1.

Paso 3: propagación hacia atrás

El error de una unidad oculta no se observa directamente. Se calcula a partir de los errores de la capa siguiente:

$$\delta_j^{(1)} = \frac{\partial R_n}{\partial a_j^{(1)}}.$$

Aplicando la regla de la cadena:

$$\delta_j^{(1)} = \sum_{c=1}^C \delta_c^{out} \frac{\partial a_c^{out}}{\partial a_j^{(1)}}.$$

Como

$$\frac{\partial a_c^{out}}{\partial a_j^{(1)}} = w_{cj}^{out} g'(a_j^{(1)}),$$

se obtiene

$$\delta_j^{(1)} = g'(a_j^{(1)}) \sum_{c=1}^C w_{cj}^{out} \delta_c^{out}.$$

Caso particular: función tangente hiperbólica

En el ejemplo se utiliza como activación oculta:

$$g(a) = \tanh(a).$$

Como

$$g'(a) = 1 - \tanh^2(a),$$

y además

$$A_j^{(1)} = \tanh(a_j^{(1)}),$$

se tiene

$$g'(a_j^{(1)}) = 1 - (A_j^{(1)})^2.$$

Por tanto,

$$\delta_j^{(1)} = \left[1 - (A_j^{(1)})^2 \right] \sum_{c=1}^C w_{cj}^{out} \delta_c^{out}.$$

Cálculo de las derivadas parciales

Una vez calculados los términos de error, las derivadas respecto a los pesos se obtienen aplicando la regla de la cadena.

Pesos entre entrada y capa oculta

$$\frac{\partial R_n}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} x_{ni}, \quad j = 1, \dots, K, \quad i = 0, \dots, p.$$

Pesos entre capa oculta y salida

$$\frac{\partial R_n}{\partial w_{cj}^{out}} = \delta_c^{out} A_j^{(1)}, \quad c = 1, \dots, C, \quad j = 0, \dots, K.$$

Estas expresiones permiten formar el gradiente $\nabla R_n(w)$.

Resumen del procedimiento

- ➊ **Propagación hacia delante:** calcular activaciones y predicción.
- ➋ **Error de salida:** obtener

$$\delta_c^{out} = f_{cn} - Y_{nc}.$$

- ➌ **Retropropagación:** calcular los errores de las unidades ocultas.
- ➍ **Gradiente:** obtener las derivadas respecto a cada peso.

Idea final

La retropropagación evita calcular todas las derivadas desde cero y reutiliza la información del error capa por capa.

Objetivo del método

El ejemplo muestra que el entrenamiento de una red neuronal se basa en dos elementos:

- Una **función de error**, que mide la diferencia entre predicción y valor real.
- Un **método eficiente para calcular gradientes**, que permite actualizar los pesos.

En este contexto, la retropropagación es fundamental porque permite calcular las derivadas necesarias para entrenar redes con muchas capas y muchos parámetros.